

Desplegament del projecte

1. Decisió tecnològica

Per al desplegament del projecte hem optat per AWS (Amazon Web Services). Aquesta decisió ens ha permès provar un entorn de producció real, similar al que es fa servir a l'empresa, i alhora aprendre a treballar amb contenidors Docker i serveis al núvol.

Tant el backend com el frontend estan desplegats en contenidors independents que s'executen a AWS ECS Fargate.

2. Backend

El backend (Symfony + PHP 8.3) està empaquetat en una imatge Docker basada en `php:8.3-apache`. La imatge inclou:

- Apache 2.4 com a servidor web
- PHP 8.3 amb les extensions necessàries (intl, opcache, pdo_mysql, zip)
- Composer per gestionar les dependències

El Dockerfile fa servir un build multistage: una primera fase amb la imatge oficial de Composer per resoldre les dependències, i una segona amb `php:8.3-apache` que copia el codi i les dependències ja optimitzades.

L'Apache està configurat amb un VirtualHost que apunta a la carpeta `public/` de Symfony i fa servir `FallbackResource /index.php` per encaminar totes les peticions al front controller de Symfony.

Les variables d'entorn sensibles (clau JWT, credencials de base de dades, etc.) s'injecten en temps d'execució des d'AWS Secrets Manager.

3. Frontend

El frontend (Vue 3 + Vite) també està empaquetat en una imatge Docker, en aquest cas amb dos stages:

- **Build stage:** una imatge de `node:20-alpine` que executa `npm ci` i `npm run build` per generar la versió de producció estàtica.
- **Serve stage:** una imatge de `httpd:2.4-alpine` (Apache) que serveix els fitxers estàtics generats.

L'Apache del frontend té un `FallbackResource /index.html` perquè Vue Router pugui gestionar les rutes des del costat client (SPA fallback).

La URL del backend s'injecta a la imatge en temps de build com a variable d'entorn VITE_API_URL.

4. Landing page

La landing està desenvolupada com una pàgina estàtica amb HTML, CSS i JavaScript vanilla. Es desplega a Amazon S3 amb hosting estàtic activat, separada del frontend Vue i del backend Symfony. Així queda accessible directament des de S3 sense necessitat de cap servidor addicional.

5. Base de dades

La base de dades és MySQL i està allotjada com a servei gestionat dins d'AWS.

6. CI/CD amb GitHub Actions

Tenim configurat un workflow de GitHub Actions que automatitza el desplegament cada vegada que es fa push a la branca main:

1. Checkout del codi
2. Login a Amazon ECR (Elastic Container Registry)
3. Build de la imatge Docker
4. Push de la imatge a ECR amb dos tags: latest i el SHA del commit
5. Actualització de la task definition d'ECS amb la nova imatge
6. Desplegament al servei d'ECS

Això ens permet desplegar canvis a producció en pocs minuts simplement fent un merge a main.

7. Resum d'infraestructura

Component	Tecnologia	Allotjament
Landing page	HTML/CSS/JS estàtic	AWS S3
Backend API	Symfony + Apache (Docker)	AWS ECS Fargate
Frontend SPA	Vue 3 build estàtic + Apache (Docker)	AWS ECS Fargate

Base de dades	MySQL	AWS
Imatges Docker	—	AWS ECR
CI/CD	GitHub Actions	GitHub

Documentació tècnica del backend

Aquest apartat recull la informació necessària per entendre l'estructura del backend i poder-lo instal·lar en local per al desenvolupament.

1. Arquitectura general

El backend està construït amb Symfony 7.4 seguint el patró MVC. Exposa una API REST que retorna JSON i, paral·lelament, un panel d'administració web fet amb Twig dins del mateix projecte. El panel d'administració permet als administradors gestionar totes les entitats (clips, jocs, usuaris, peticions), generar i revocar API Keys, i interactuar amb un chatbot d'ajuda integrat amb OpenRouter.

L'estructura principal del codi és:

src/

- |— Controller/ — Controladors (API i panel admin)
- |— Entity/ — Entitats de Doctrine (mapping de taules)
- |— Repository/ — Repositoris (consultes a la BD)
- |— Security/ — Autenticadors personalitzats (JWT i API Key)
- |— Form/ — Formularis del panel admin

config/

- |— packages/ — Configuració dels bundles
- |— jwt/ — Clau pública JWT

migrations/ — Migracions de Doctrine

templates/ — Plantilles Twig del panel admin

5. Desplegament en producció

El desplegament es fa amb Docker i AWS ECS. Cada push a la branca `main` activa el workflow de GitHub Actions que construeix la imatge, la puja a ECR i actualitza el servei d'ECS. Veure l'apartat de Desplegament per als detalls.

6. Estructura de l'API

Tots els endpoints de l'API estan sota el prefix `/failrun/api/`. La documentació pública i interactiva amb tots els paràmetres i respostes està disponible amb Swagger a la ruta `/api/doc`.

7. Estratègia de control de versions

Hem treballat amb les següents branques seguint una versió simplificada de GitFlow:

- `main` — versió de producció. Cada push aquí activa el desplegament automàtic.
- `develop` — branca d'integració del desenvolupament.
- `feature/*` — branques per al desenvolupament de funcionalitats noves.

El flux habitual és: crear una branca `feature/` des de `develop`, treballar-hi, fer merge a `develop` quan està llest, i finalment fer merge a `main` quan toca desplegar a producció.

8. Panel d'administració

A més de l'API REST, el backend exposa un panel d'administració accessible a `/failrun/admin/login`. Aquest panel està fet amb Twig dins del mateix projecte Symfony i utilitza autenticació per sessió (no JWT).

Permet als administradors:

- Gestionar totes les entitats (clips, jocs, usuaris, valoracions, peticions)
- Generar i revocar API Keys per a serveis externs
- Interactuar amb un chatbot IA d'ajuda integrat amb OpenRouter